
MODULE *FeedSync*

EXTENDS *Integers, Sequences, FiniteSets, TLC*

CONSTANTS

Packages, Set of packages to be published *e.g.*, $\{“pkgA”, “pkgB”\}$
MaxTime Maximum logical time *e.g.*, 3

VARIABLES

published_feeds, Feeds published by the publisher
mirror_tip, The latest feed state accepted by the mirror
mirror_pkgs, Set of packages successfully mirrored
mode, “Timestamp” or “Merkle”
pub_time, Publisher’s local clock
pub_tip_cid Publisher’s latest *CID*

vars $\triangleq \langle published_feeds, mirror_tip, mirror_pkgs, mode, pub_time, pub_tip_cid \rangle$

Init $\triangleq$

$\wedge published_feeds = \{\}$
 $\wedge mirror_tip = [time \mapsto 0, cid \mapsto 0, prev \mapsto 0]$
 $\wedge mirror_pkgs = \{\}$
 We explicitly set mode to “Merkle” here because “Timestamp”
 is intentionally vulnerable to liveness failures (out-of-order packets).
 To see the counter-example, change this back to $\{“Timestamp”, “Merkle”\}$.
 $\wedge mode = “Merkle”$
 $\wedge pub_time = 0$
 $\wedge pub_tip_cid = 0$

PUBLISHER ACTIONS

The publisher creates a perfectly valid linear chain of updates.

Publish(*pkg*, *new_cid*) $\triangleq$
 $\wedge \neg \exists f \in published_feeds : f.pkg = pkg$
 $\wedge pub_time < MaxTime$
 $\wedge LET\ t \triangleq pub_time + 1$
 IN
 $\wedge pub_time' = t$
 $\wedge published_feeds' = published_feeds \cup \{[pkg \mapsto pkg, time \mapsto t, cid \mapsto new_cid, prev \mapsto pub_tip_cid]\}$
 $\wedge pub_tip_cid' = new_cid$
 $\wedge UNCHANGED \langle mirror_tip, mirror_pkgs, mode \rangle$

MIRROR ACTIONS

$$\begin{aligned}
& \text{ReceiveTimestamp}(\text{feed}) \triangleq \\
& \quad \wedge \text{mode} = \text{"Timestamp"} \\
& \quad \wedge \text{feed.time} > \text{mirror_tip.time} \\
& \quad \wedge \text{mirror_tip}' = [\text{mirror_tip} \text{ EXCEPT } !.\text{time} = \text{feed.time}] \\
& \quad \wedge \text{mirror_pkgs}' = \text{mirror_pkgs} \cup \{\text{feed.pkg}\} \\
& \quad \wedge \text{UNCHANGED } \langle \text{published_feeds}, \text{mode}, \text{pub_time}, \text{pub_tip_cid} \rangle \\
\\
& \text{ReceiveMerkle}(\text{feed}) \triangleq \\
& \quad \wedge \text{mode} = \text{"Merkle"} \\
& \quad \wedge \text{feed.prev} = \text{mirror_tip.cid} \\
& \quad \wedge \text{mirror_tip}' = [\text{mirror_tip} \text{ EXCEPT } !.\text{cid} = \text{feed.cid}] \\
& \quad \wedge \text{mirror_pkgs}' = \text{mirror_pkgs} \cup \{\text{feed.pkg}\} \\
& \quad \wedge \text{UNCHANGED } \langle \text{published_feeds}, \text{mode}, \text{pub_time}, \text{pub_tip_cid} \rangle \\
\\
& \text{Receive} \triangleq \\
& \quad \exists \text{feed} \in \text{published_feeds} : \\
& \quad \quad \text{IF } \text{mode} = \text{"Timestamp"} \text{ THEN } \text{ReceiveTimestamp}(\text{feed}) \text{ ELSE } \text{ReceiveMerkle}(\text{feed}) \\
\\
& \text{Prevent deadlock when system reaches a quiescent state} \\
& \text{Stutter} \triangleq \\
& \quad \wedge \text{UNCHANGED } \text{vars}
\end{aligned}$$

NEXT STATE RELATION

$$\begin{aligned}
\text{Next} \triangleq \\
& \quad \vee \exists \text{pkg} \in \text{Packages}, \text{cid} \in 1 \dots 3 : \text{Publish}(\text{pkg}, \text{cid}) \\
& \quad \vee \text{Receive} \\
& \quad \vee \text{Stutter}
\end{aligned}$$

FAIRNESS & PROPERTIES

Structural Integrity (Safety Invariant)

$$\begin{aligned}
\text{TypeOK} \triangleq \\
& \quad \wedge \text{mirror_pkgs} \subseteq \text{Packages} \\
& \quad \wedge \text{mode} \in \{\text{"Timestamp"}, \text{"Merkle"}\} \\
& \quad \wedge \text{pub_time} \in 0 \dots \text{MaxTime} \\
& \quad \wedge \text{pub_tip_cid} \in 0 \dots 3 \\
& \quad \wedge \text{mirror_tip} \in [\text{time} : 0 \dots \text{MaxTime}, \text{cid} : 0 \dots 3, \text{prev} : 0 \dots 3] \\
& \quad \wedge \forall f \in \text{published_feeds} : \\
& \quad \quad f \in [\text{pkg} : \text{Packages}, \text{time} : 1 \dots \text{MaxTime}, \text{cid} : 1 \dots 3, \text{prev} : 0 \dots 3]
\end{aligned}$$

Weak fairness ensures that if an action is continuously enabled, it eventually fires.

$$\text{Fairness} \triangleq$$

$$\begin{aligned} & \wedge \forall pkg \in Packages, cid \in 1 \dots 3 : WF_{vars}(Publish(pkg, cid)) \\ & \wedge WF_{vars}(Receive) \end{aligned}$$

Liveness Property: Eventually, all published packages should be mirrored.

$$\begin{aligned} EventuallyConsistent & \triangleq \\ & (\forall p \in Packages : \exists f \in published_feeds : f.pkg = p) \leadsto (Packages \subseteq mirror_pkgs) \end{aligned}$$

$$Spec \triangleq Init \wedge \Box[Next]_{vars} \wedge Fairness$$
